

MergeRank

Competitive Programming Analytics & Mentorship Platform

Full Project Plan | MERN Stack

LeetCode • GitHub • Codeforces • CodeChef • HackerRank

5

Platforms

2

User Roles

**Real
Deploy**
Scope

✓

AI

1. Problem Statement

Students solving competitive programming problems on platforms like LeetCode face a common morale problem: global ranks in the hundreds of thousands feel discouraging, even when genuine progress is being made. At the same time, mentors and faculty have no centralized way to track student activity across multiple platforms — they resort to manually visiting each student's profile, which is time-consuming and unscalable.

Student Pain Points	Mentor Pain Points
Global LeetCode rank in lakhs → low morale	No dashboard to see all students' progress
No peer comparison within their own batch	Manual profile-by-profile checking
Don't know what topics to focus on next	Hard to give placement recommendations fairly
Activity scattered across 5+ platforms	No insight into which students need help

2. Solution — MergeRank

MergeRank is a web application where students register their usernames from multiple competitive programming platforms. The app aggregates their data via APIs and presents two distinct dashboards: one for students (peer-level comparisons, topic analysis, AI suggestions) and one for mentors (class-wide analytics, placement readiness scores, weak area alerts).

Core Value Proposition

- Students are ranked only within their batch/class — not against the global population
- Mentors get a single dashboard replacing hours of manual checking
- AI suggests the next topic/problem type based on each student's weak areas
- All 5 platforms unified into one profile: LeetCode, GitHub, Codeforces, CodeChef, HackerRank

3. Feature List

3.1 Student Features

Profile & Platform Integration

- Enter usernames for LeetCode, GitHub, Codeforces, CodeChef, HackerRank
- Auto-fetch and sync data periodically (every 6–12 hours via cron job)
- Manual 'Sync Now' button
- Unified stats card: total problems solved, contest rating, streak, repos

Peer Comparison (Key Differentiator)

- Leaderboard scoped to your batch/section/college only
- Compare by: problems solved, contest rating, GitHub commits, streak
- Filter leaderboard by platform, topic category (Arrays, DP, Graphs, etc.)
- See your percentile within your batch (not global rank)
- Anonymous mode: compare without revealing identity to peers

Personal Analytics

- Topic-wise breakdown: how many problems solved per DSA topic
- Difficulty distribution: Easy / Medium / Hard ratio
- Streak calendar (GitHub-style heatmap) of activity
- Contest history graph — rating over time
- Platform consistency score — how regularly they code

AI-Powered Suggestions

- Weak area detection: identifies topics with lowest solve rate
- Suggests 3–5 specific problems to solve next (from LeetCode/CF)
- Recommends topics to study based on placement trends
- Weekly goal suggestions personalized to current skill level

3.2 Mentor Features

Class Dashboard

- See all students in one table: solved count, rating, streak, last active
- Sort & filter by any metric
- Color-coded performance indicators (green / yellow / red)
- Export to CSV/PDF for reports

Placement Readiness

- Auto-calculated placement readiness score per student (0–100)
- Based on: problems solved, difficulty ratio, contest participation, GitHub activity
- Quick-view: 'Placement Ready', 'Needs Improvement', 'At Risk' tags
- One-click to view full student profile

Alerts & Recommendations

- Alert: Students who haven't been active for 7+ days
- Alert: Students consistently failing at a specific topic category
- Mentor can add personal notes on any student
- Mentor can send in-app nudges/messages to students

Batch Management

- Create and manage multiple batches (e.g., 2025-CS-A, 2026-IT-B)
- Add/remove students from batches
- Invite students via email or enrollment code
- Compare two batches side by side

4. Tech Stack

Layer	Technology	Purpose
Frontend	React.js + Vite	SPA, dashboards, charts
	Tailwind CSS	Responsive UI styling
	Recharts / Chart.js	Analytics graphs & heatmaps
Backend	Node.js + Express.js	REST API server
	node-cron	Scheduled data sync jobs
	Axios	API calls to platforms
Database	MongoDB + Mongoose	User data, stats, batches
Auth	JWT + bcrypt	Secure role-based auth
	Google OAuth (optional)	College email login
AI	OpenAI API / Gemini	AI suggestions engine
Deploy	Render / Railway (backend)	Free tier, real deployment
	Vercel (frontend)	Fast CDN delivery
	MongoDB Atlas	Cloud DB (free tier)

5. Platform Data Fetching Strategy

Each platform is handled differently based on what it exposes publicly. Here is the exact approach for each:

Platform	Method	Data Available	Notes
LeetCode	Unofficial GraphQL API	Problems solved, difficulty, streak, contest rating, badges	No official key needed. Use leetcode-stats-api or direct GQL
GitHub	Official REST API v3	Repos, commits, stars, languages, contribution graph	Free with 60 req/hr unauthenticated, 5000 with token
Codeforces	Official Public API	Rating, rank, solved problems, contest history	Completely free, no auth needed
CodeChef	Web scraping (cheerio)	Rating, stars, problems solved, contests	No official API — scrape public profile page
HackerRank	Unofficial API / scraping	Badges, certificates, solved count, leaderboard	Profile is public; use profile JSON endpoint

6. Database Schema (MongoDB)

6.1 User Collection

Stores both students and mentors with role-based fields.

```
{  _id, name, email, password (hashed), role: 'student' | 'mentor',  college, batch, platforms: {    leetcode: { username, totalSolved, easy, medium, hard, rating, streak, lastSynced },    github: { username, repos, totalCommits, stars, topLanguages, lastSynced },    codeforces: { username, rating, rank, solved, lastSynced },    codechef: { username, rating, stars, solved, lastSynced },    hackerrank: { username, badges, certificates, solved, lastSynced }  },  aiSuggestions: [ { topic, problems[], generatedAt } ],  createdAt, updatedAt }
```

6.2 Batch Collection

```
{  _id, name, mentorId (ref: User), students: [userId], inviteCode, college, year, createdAt }
```

6.3 MentorNote Collection

```
{  _id, mentorId, studentId, note (text), createdAt }
```

6.4 SyncLog Collection

```
{  _id, userId, platform, status: 'success'|'failed', message, timestamp }
```


7. Backend API Routes

Authentication

Method	Route	Description
POST	/api/auth/register	Register student or mentor
POST	/api/auth/login	Login, returns JWT
GET	/api/auth/me	Get current user profile

Student Routes

Method	Route	Description
PUT	/api/student/platforms	Save/update platform usernames
POST	/api/student/sync	Manually trigger data sync for all platforms
GET	/api/student/stats	Get own aggregated stats
GET	/api/student/leaderboard/:batchId	Get batch leaderboard
GET	/api/student/suggestions	Get AI-generated suggestions

Mentor Routes

Method	Route	Description
GET	/api/mentor/batches	Get all batches created by mentor
POST	/api/mentor/batches	Create new batch
GET	/api/mentor/batch/:id/students	Get all students + stats in a batch
GET	/api/mentor/student/:id	Get full profile of one student
POST	/api/mentor/notes	Add note on a student
GET	/api/mentor/alerts/:batchId	Students inactive 7+ days or struggling
GET	/api/mentor/export/:batchId	Export batch stats as CSV

8. AI Suggestion Engine

The AI engine analyses a student's topic-wise solve distribution from LeetCode and Codeforces, detects weak areas, and generates targeted suggestions using an LLM prompt.

How It Works

1. Collect student's topic-wise data: e.g., Arrays: 45 solved, DP: 3 solved, Graphs: 1 solved
2. Sort topics by solve count ascending → weakest topics identified
3. Construct a prompt to OpenAI / Gemini API with student's weak topics + difficulty level
4. Parse response → extract 3–5 specific problem recommendations + study tip
5. Cache result in DB (aiSuggestions field), refresh weekly or on demand

Sample Prompt Template

```
You are a competitive programming coach. A student has the following LeetCode topic stats:
Arrays: 45 | Strings: 30 | DP: 4 | Graphs: 2 | Trees: 8 | Binary Search: 5
Their current level: Medium problems (LeetCode rating ~1400).
Suggest: 1. Top 2 weakest topics to focus on
2. 3 specific LeetCode problem names to solve
3. One study resource or approach for each weak topic
Keep it concise and actionable.
```

Placement Readiness Score Formula

Score = (Problems Solved × 0.35) + (Contest Rating × 0.25) + (Hard Ratio × 0.20) + (GitHub Activity × 0.10) + (Streak × 0.10)

Each component is normalized to 0–100 before applying weights. Score thresholds: 75+ = Placement Ready, 50–74 = Needs Improvement, <50 = At Risk.

Backend (Node.js + Express)

Frontend (React + Vite)

```

graph LR
    frontend[frontend/] --- public[public/]
    frontend --- functions[functions]
    public --- src[src/]
    public --- components[components/]
    src --- api[api/]
    src --- charts[charts/]
    components --- cards[cards/]
    components --- common[common/]
    api --- axios[→ axios instances & API call]
    charts --- heatmap[→ Heatmap, BarChart, RadarChart, LineChart]
    cards --- platform[→ PlatformCard, StatCard, AlertCard]
    common --- student[→ student/]
    common --- dashboard[→ Dashboard, Leaderboard, Suggestions, Profile]
    student --- mentor[→ mentor/]
    student --- class[→ ClassDashboard, StudentProfile]
    mentor --- auth[→ AuthContext.jsx]
    hooks[→ hooks/]
    context[→ context/]
    useStats[→ useStats.js, useLeaderboard.js]
    utils[→ utils/]
    formatters[→ formatters.js, scoreUtils.js]
    app[→ App.jsx, main.jsx]
    vite[→ vite.config.js]
  
```

10. Development Roadmap

Phase	Duration	Tasks	Milestone
Phase 1	Week 1–2	Project setup, MongoDB models, JWT auth, register/login API + React auth pages	Auth working end-to-end
Phase 2	Week 3–4	Platform service files: LeetCode GQL, GitHub API, Codeforces API. Manual sync endpoint. Display stats on student dashboard.	3 platforms pulling real data
Phase 3	Week 5	CodeChef + HackerRank scrapers. Cron job for auto-sync. SyncLog tracking.	All 5 platforms live
Phase 4	Week 6–7	Batch system, mentor dashboard, class table, student detail view, mentor notes, CSV export.	Mentor portal complete
Phase 5	Week 8	Leaderboard (batch-scoped), percentile calculation, comparison filters, charts (heatmap, radar, line).	Leaderboard + analytics
Phase 6	Week 9	AI integration: weak area detection, OpenAI/Gemini prompt, display suggestions. Placement readiness score.	AI suggestions live
Phase 7	Week 10	UI polish, mobile responsiveness, error handling, testing, deploy to Vercel + Render + MongoDB Atlas.	Deployed & demo-ready

11. What Makes MergeRank Unique

Feature	Why It's Unique / Not Done Before
Batch-scoped leaderboard	Students compare only within their class — morale stays high instead of being crushed by global ranks
5-platform aggregation	No existing tool unifies LeetCode + GitHub + CF + CC + HR in one dashboard
Mentor placement readiness score	Auto-calculated score helps mentors make fair placement recommendations without bias
AI topic suggestions	Personalized weak-area detection with actual problem recommendations — not generic advice
Inactive student alerts	Mentors get notified before a student falls behind — proactive not reactive
Invite code system	Mentors share a code → students join instantly. Clean onboarding for real college deployment

12. Suggested Project Name & Tagline

Name	Tagline	Domain Idea
MergeRank	Track your growth. Prove your potential.	MergeRank.vercel.app
RankWise	Compete smarter. Rank within your world.	rankwise.vercel.app
DevPulse	Your coding heartbeat. Visible to mentors.	devpulse.vercel.app

Built with MERN Stack • Real Deployment • AI-Powered
MongoDB • Express.js • React.js • Node.js